CI/CD & GitHub Actions

WE MERGE, THE ROBOT DOES THE REST

BLUE = REPLACE WITH YOUR OWN VALUE

01 CI, CD… and CD

Continuous Integration
Every push and PR is **built and tested** automatically.

Continuous Delivery
The code is always ready to ship; a human clicks "deploy".

Continuous Deployment
Green on `main` = live, with no human step.

A **pipeline** = the chain of automatic steps between your commit and production: **build** → **test** → **package** → **deploy**.

02 The vocabulary

Workflow	A YAML file in `.github/workflows/`. One pipeline.
Event	What starts it: `push`, `pull_request`, a schedule…
Job	A group of steps on **one machine**. Jobs run in parallel by default.
Step	One command (`run:`) or one action (`uses:`).
Action	A reusable brick, e.g. `actions/checkout`. Thousands on the Marketplace.
Runner	The machine that runs a job (`ubuntu-latest`, windows, macos, or your own).
Artifact	A file produced by a job and kept (build, report…).
Secret	A password or token stored encrypted in **Settings → Secrets**.
Environment	`staging`, `production`… with their own secrets and approvals.

03 When does it run? `on:`

`push: branches: [main]`	On every push to **main**.
`pull_request:`	On every PR: perfect for checks.
`workflow_dispatch:`	A **"Run workflow"** button in the Actions tab.
`schedule: - cron: "0 7 * * 1"`	On a schedule (here every Monday 07:00 UTC).
`release: types: [published]`	When you publish a Release (tag).
`paths: ["src/**"]`	Only if those files changed.

04 Anatomy of a workflow

```yaml
# .github/workflows/ci.yml
name: ci                          # shown in the Actions tab
on:                               # WHEN
  push:
    branches: [main]
  pull_request:
permissions:
  contents: read                  # least privilege
jobs:
  test:                           # a job = one machine
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4   # get the code
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci                 # install
      - run: npm test               # fail = red ❌
  deploy:
    needs: test                   # only if test is green
    if: github.ref == 'refs/heads/main'
    environment: production
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh
        env:
          TOKEN: ${{ secrets.DEPLOY_TOKEN }}
```

YAML is **indentation-sensitive**: 2 spaces, never tabs.

05 Useful building blocks

`needs: [build, test]`	Runs a job **after** others (otherwise in parallel).
`strategy: matrix: node: [18, 20, 22]`	Same job on several versions / OSes.
`${{ secrets.NAME }}`	Reads a secret (hidden in the logs).
`${{ github.sha }} · ${{ github.actor }}`	Contexts: info about the run (commit, who…).
`actions/upload-artifact@v4`	Keeps a file from the job (download it from the run page).
`concurrency: deploy`	Never two deployments at the same time.

See a real pipeline
The Builders Wall: `.github/workflows/check.yml` (PR) and `deploy.yml` (main → GitHub Pages)
`github.com/alibenyezza/dvb-git-workshop`

06 Our pipeline, step by step

`PR` → `check: validate` → `merge` → `deploy: build` → `Pages` → `live`

A **ruleset** on `main` requires a PR and a green `validate` check: nothing broken can reach production.

07 Good practices & security

- **Pin your actions**: `@v4`, or a full commit SHA for third-party actions.
- **Least privilege**: set `permissions:` to what the job really needs.
- **Never a secret in the code**: use Secrets, and rotate any secret that leaked.
- **Protect `main`**: required PR + required checks (rulesets).
- **Approval for production**: an environment with required reviewers.
- **Keep it fast**: cache dependencies, run jobs in parallel, fail early.
- **Dependabot** keeps your actions and dependencies up to date.

08 It's red ❌: debug it

`Actions tab → the run → the job`	Open the red step: the error is in its log.
`"Re-run jobs"`	Re-runs a failed job (useful for a network glitch).
`ACTIONS_STEP_DEBUG = true`	As a repo variable or secret: very detailed logs.
`actionlint`	Finds YAML and syntax errors before you push.
`act`	Runs your workflows locally, with Docker.

09 Practise on your own projects

- Add a CI that runs your **tests and linter** on every PR.
- Deploy your **portfolio** with GitHub Pages (`you.github.io`).
- Add the **status badge** of your workflow to your README.
- Publish a **Release** automatically when you push a tag.

10 Watch 🎬

- ▶ **GitHub Actions Tutorial** ↗ `32 MIN`
  Concepts + a CI/CD pipeline with Docker · *TechWorld with Nana*
- ▶ **From Zero to Hero in 90 minutes** ↗ `1 H 25`
  Environments, secrets, runners · *CoderDave*
- ▶ **Complete GitHub Actions Course** ↗ `BEGINNER → PRO`
  DevOps Directive
- ▶ **5 Ways to DevOps-ify your App** ↗ `SHORT`
  Fast, fun automation ideas · *Fireship*
- ▶ **GitHub Actions Certification course** ↗ `≈ 3 H`
  Full GH-200 exam prep · *freeCodeCamp*

11 Learn by doing FREE

- 🧪 **GitHub Skills** ↗
  Hands-on courses in a repo: *Hello GitHub Actions*, *Test with Actions*, *Reusable workflows*...
- 📘 **Microsoft Learn: GitHub Actions path** ↗
  The official learning path for the GH-200 exam
- 📖 **GitHub Actions docs** ↗
  Quickstart, workflow syntax, events, secrets, security
- 🧩 **Actions Marketplace** ↗ • **DevOps roadmap** ↗
  Ready-made actions · what to learn next (Docker, cloud...)

12 Get certified 🎓

EXAM	CERTIFICATION	FOR
GH-900	**GitHub Foundations** FREE	Start here
GH-200	**GitHub Actions**	CI/CD, intermediate
GH-300	**GitHub Copilot**	AI-assisted coding
GH-500	**GitHub Advanced Security**	Secure code
GH-100	**GitHub Administration**	Admins

Students: verify your status on GitHub Education, then claim a **free GH-900 voucher** from the Student Developer Pack (while supplies last). Exams are booked on Microsoft Learn (Pearson VUE, online), and each page has a **free practice assessment**.

All the links, clickable
In the workshop repo: `resources/`
Student Developer Pack: `education.github.com/pack`